

Experimental Evaluation of Idempotency, Retry Mechanisms, and Concurrency Control in Webhook Processing

Jagjit Kaur | Independent experimental study | October 2026

Abstract

This study evaluates selected reliability mechanisms for webhook processing using a seeded SQLite simulation and a local FastAPI/PostgreSQL prototype. The simulation compares baseline handling, event-ID deduplication, and immediate retry of injected transient failures. The PostgreSQL implementation is assessed using integration tests, in-process HTTP timing, sequential processing benchmarks, and a deliberately synchronized concurrency experiment. Across five SQLite repetitions, average duplicate effects decreased from 44.2 to zero with deduplication, while retry-enabled processing completed all 200 unique effects under the specified failure model. In the concurrency experiment, five workers without explicit read locking entered the processing branch, versus one worker with row-level locking. The results characterize these local workloads and do not demonstrate exactly-once external effects or production throughput.

1. Research questions

RQ1. How do event-ID deduplication and immediate retries affect duplicate effects and completed unique events under simulated duplicate delivery and transient pre-effect failure?

RQ2. What functional behavior and local request latencies are observed in a PostgreSQL-backed webhook prototype?

RQ3. How does explicit PostgreSQL row-level locking affect processing claims in a controlled same-event contention scenario?

2. Experimental design

SQLite simulation. Each of five seeded repetitions contains 200 unique event identifiers, 50 duplicate deliveries, and transient pre-effect failures assigned to 10% of identifiers. Three strategies are evaluated: baseline processing, idempotent processing, and idempotent processing with up to three immediate attempts. The simulation uses an in-memory SQLite database; it is not an HTTP deployment.

PostgreSQL prototype. A FastAPI API backed by SQLAlchemy and local PostgreSQL stores webhook events with a unique event-ID constraint. The prototype provides duplicate detection, deterministic first-attempt failure simulation, manual retry endpoints, and a processing path using `SELECT FOR UPDATE`. Seven API integration tests passed; a separate two-test comparison confirmed basic single-event operation of both processing implementations.

Timing protocols. Six sequential TestClient runs each submit 100 new events, 20 duplicate deliveries, and 100 process requests. A separate sequential benchmark records 100 processing events per implementation per repetition, across five repetitions (1,000 measurements). Its corrected timing window begins before the first database lookup in both modes.

Concurrency protocol. Five workers target the same event in each of five repetitions per mode. In the no-lock mode, a barrier deliberately synchronizes workers after reading the initial state. In the locked mode, `SELECT FOR UPDATE` serializes access. This asymmetry is intentional for race demonstration and prevents interpreting the experiment as a balanced throughput test.

3. Results: SQLite simulation

Strategy	Successful deliveries	Failed deliveries	Unique effects	Duplicate effects
Baseline	230	20	185.8	44.2
Idempotent	230	20	185.8	0
Idempotent + retry	250	0	200	0

Values are means over five seeded repetitions. Deduplication eliminated repeated effects in this model, and immediate retries recovered the injected transient pre-effect failures. These counts are simulation outcomes, not observed external side effects in PostgreSQL.

4. Results: FastAPI/PostgreSQL HTTP pilot

Operation	Requests per run	Mean of run means (ms)	Range of run means (ms)
Receive unique	100	5.744	4.816–7.026
Receive duplicate	20	4.796	3.288–5.792
Process event	100	8.677	6.750–9.957

The six runs contain 1,320 measured requests. Latencies include TestClient request handling and local database operations but exclude external network transit. Differences among operations are descriptive; no production throughput or statistical significance is inferred.

5. Results: sequential processor comparison

Implementation	Observations	Mean latency (ms)	Median latency (ms)
With explicit row lock	500	4.876	4.923
Without explicit row lock	500	3.816	3.718

The locked path had a higher observed mean in this sequential dataset. It also performs an initial event lookup followed by a locked lookup, whereas the no-lock path performs one lookup. Consequently, the result compares complete implementation paths rather than isolating SELECT FOR UPDATE overhead. The earlier benchmark with unequal timing boundaries is preserved separately as exploratory data and is not used for this comparison.

6. Results: controlled concurrency demonstration

Implementation	Repetitions	Workers claiming processing	Final stored attempt count
Without explicit read lock	5	5 of 5 per repetition	1
With SELECT FOR UPDATE	5	1 of 5 per repetition	1

In the deliberately synchronized no-lock branch, all five workers read an unprocessed event and entered the processing branch; the stored attempt count remained one, consistent with a lost-update pattern. With

row-level locking, one worker entered that branch and the remaining workers observed a processed event. The claims count is not a measurement of independent payment, email, or other external effects.

7. Discussion and limitations

The simulation and prototype address different layers of webhook reliability. Event-ID uniqueness prevents duplicate stored rows in the tested PostgreSQL requests; it does not alone ensure a business action executes once. Row-level locking serializes the tested same-row processing decisions, while immediate retries in the SQLite model recover only the defined transient failure class. The PostgreSQL retry endpoint is manually invoked; automatic scheduling, backoff, dead-letter handling, transactional outbox behavior, crash recovery, external side effects, and broad production load testing were outside the study scope.

Further limitations include a local single-machine environment, small and partly synthetic workloads, timing variation, different query patterns in the sequential comparison, and deliberately asymmetric concurrency synchronization. These findings should not be generalized to exactly-once delivery or production reliability guarantees.

8. Reproducibility

The project archive contains application code (api/), experimental scripts (src/), tests (tests/), raw and summarized CSV data (results/), and notes (docs/). PostgreSQL experiments require a local database, a private DATABASE_URL configuration, and schema columns for attempt_count and last_error. Running the experiments inserts new rows; use a disposable research database. Representative commands are listed below.

```
python -m pytest tests -v
python -m src.postgres_experiment
python -m src.processor_benchmark
python -m src.concurrency_experiment
```

9. Conclusion

Within the defined SQLite model, deduplication removed duplicate effects and retries recovered the injected pre-effect failures. In the local PostgreSQL prototype, the tested uniqueness constraint prevented duplicate event rows, and row-level locking restricted branch-entry claims to one worker in the constructed contention scenario. These findings support a limited, reproducible investigation of webhook reliability mechanisms rather than a claim of production exactly-once processing.

References and technical documentation

- [1] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: Explicit Locking. <https://www.postgresql.org/docs/16/explicit-locking.html>
- [2] PostgreSQL Global Development Group. PostgreSQL 16 Documentation: INSERT. <https://www.postgresql.org/docs/16/sql-insert.html>
- [3] FastAPI. Testing. <https://fastapi.tiangolo.com/tutorial/testing/>
- [4] SQLite. Transaction documentation. https://www.sqlite.org/lang_transaction.html